

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



LexiParse

PONDERACIÓN: 40 pts

 **Tiempo estimado: 45 hrs**

Índice

1. MARCO FORMATIVO.....	3
1.1. Valor	3
1.2. Competencia(s).....	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje.....	3
2.1. Objetivo SMART	3
3. Resumen Ejecutivo.....	4
4. Enunciado del Proyecto.....	4
4.1 Descripción del problema o necesidad a resolver	4
4.2 Alcance del proyecto	4
4.3 Recursos y herramientas a utilizar	4
4.4 Entregables	4
5. Material de apoyo	5
6. Metodología.....	5
7. Cronograma.....	6
8. Rúbrica de Calificación.....	6
8.1 Requisitos para optar a la calificación	6
8.2 Detalle de la Calificación	7
8.3 Comentarios Generales	8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad	El estudiante debe planificar, desarrollar y probar cada fase del intérprete con orden, documentando resultados y corrigiendo errores oportunamente.

1.2. Competencia(s)

- Con la elaboración de este proyecto el estudiante fortalecerá la competencia de analizar, diseñar e implementar soluciones de software basadas en principios de lenguajes formales, compiladores e interfaces de usuario, integrando teoría y práctica en una aplicación funcional de escritorio o similar

1.3. Habilidad(es) blandas a formar

- Pensamiento analítico para definir tokens, reglas gramaticales y estructuras válidas del lenguaje.
- Resolución de problemas para depurar errores léxicos, sintácticos y de integración.
- Organización del trabajo para dividir el proyecto en fases y cumplir con entregables parciales.
- Comunicación técnica para documentar reglas, decisiones de diseño y funcionamiento del intérprete.
- Autonomía en la investigación y selección de herramientas de implementación.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Diseñar e implementar un intérprete para un lenguaje de programación propio que incluya análisis léxico, análisis sintáctico e interfaz gráfica.	El proyecto se considerará cumplido al reconocer correctamente tokens, validar estructuras sintácticas, mostrar árboles sintácticos y reportar errores en al menos 10 casos de prueba.	Mediante la definición formal del lenguaje, la implementación de analizadores, pruebas unitarias y una interfaz gráfica para visualizar resultados.	Para aplicar los conceptos de compiladores en una solución práctica y fortalecer habilidades de diseño de lenguajes, programación y validación de entradas.	El proyecto deberá estar finalizado, documentado y presentado en la fecha establecida por el curso durante el segundo semestre de 2026.

3. Resumen Ejecutivo

El proyecto consiste en desarrollar un intérprete para un lenguaje de programación diseñado por el estudiante. La solución deberá contemplar dos fases esenciales de un compilador: la fase léxica y la fase sintáctica. En la fase léxica se definirán y reconocerán los tokens del lenguaje, tales como palabras reservadas, operadores, identificadores, delimitadores y literales. En la fase sintáctica se construirá una gramática libre de contexto que permita validar estructuras del lenguaje, como declaraciones, expresiones y estructuras de control.

Además de los analizadores, el sistema deberá incorporar una interfaz gráfica que permita ingresar código fuente, ejecutar el análisis y visualizar de forma clara los tokens reconocidos, el árbol sintáctico generado y los errores léxicos o sintácticos encontrados. El propósito del proyecto es que el estudiante aplique fundamentos de compiladores y lenguajes formales en un producto funcional, usable y técnicamente bien estructurado.

4. Enunciado del Proyecto

El estudiante deberá diseñar e implementar un intérprete para un lenguaje de programación propio, aplicando los conceptos fundamentales de análisis léxico y análisis sintáctico. El proyecto iniciará con la definición del lenguaje, estableciendo sus tokens, palabras reservadas, operadores, identificadores, literales y signos especiales. Posteriormente, se desarrollará un analizador léxico que procese el código fuente, identifique cada componente válido del lenguaje y detecte errores léxicos.

Con base en la especificación del lenguaje, el estudiante deberá definir una gramática libre de contexto que describa formalmente la estructura sintáctica del lenguaje propuesto. A partir de esta gramática, implementará un analizador sintáctico capaz de validar secuencias de tokens, detectar errores de sintaxis y generar un árbol sintáctico o árbol de derivación para las entradas válidas.

Como parte de la solución integral, el proyecto deberá incluir una interfaz gráfica que facilite la interacción con el usuario. Esta interfaz permitirá ingresar o cargar código fuente, ejecutar el análisis, mostrar la lista de tokens, visualizar el árbol sintáctico y presentar mensajes de error de manera comprensible. El lenguaje propuesto podrá ser sencillo, pero debe ser lo suficientemente consistente para soportar expresiones, declaraciones y al menos una estructura de control.

4.1 Descripción del problema o necesidad a resolver

- En el estudio de compiladores y lenguajes formales, con frecuencia los conceptos de análisis léxico y sintáctico se abordan de forma teórica, lo que dificulta que el estudiante comprenda su aplicación práctica dentro de herramientas reales de procesamiento de lenguajes. Existe, por tanto, la necesidad de desarrollar un proyecto integrador que permita aplicar dichos conceptos en una implementación funcional.

Este proyecto busca resolver esa necesidad mediante la construcción de un intérprete básico que evidencie cómo un lenguaje de programación puede definirse formalmente, analizarse y validarse. De esa forma, el estudiante no solo comprenderá la teoría detrás de los compiladores, sino que también desarrollará una solución capaz de reconocer estructuras válidas, detectar errores y presentar resultados de manera visual e interactiva.

4.2 Alcance del proyecto

Alcance obligatorio:

- Definir un lenguaje de programación propio con un conjunto claro de tokens.
- Incluir como mínimo palabras reservadas, identificadores, números, operadores aritméticos o relacionales, delimitadores y asignación.
- Implementar un analizador léxico que reconozca tokens válidos y reporte errores léxicos.
- Diseñar una gramática libre de contexto para el lenguaje propuesto.
- Implementar un analizador sintáctico que valide entradas según la gramática definida.
- Soportar al menos expresiones, declaraciones y una estructura de control, por ejemplo: if, while o for.
- Generar y mostrar un árbol sintáctico o árbol de derivación para entradas válidas.
- Mostrar errores sintácticos de forma clara.
- Desarrollar una interfaz gráfica para ingresar código fuente y visualizar tokens, árbol sintáctico y errores.
- Entregar documentación técnica y manual de uso.

Alcance opcional:

- Agregar análisis semántico básico, como validación de variables declaradas.
- Ejecutar instrucciones del lenguaje mediante un intérprete simple.
- Permitir abrir y guardar archivos de código fuente.
- Exportar los tokens o el árbol sintáctico a archivo.
- Incorporar resaltado de sintaxis en la interfaz.
- Agregar soporte para más estructuras del lenguaje, como funciones, comentarios o tipos de datos adicionales.

4.3 Recursos y herramientas a utilizar

Tipo	Categoría	Descripción
Obligatorio	Software	Lenguaje de programación para la implementación, por ejemplo Python, Java o C++.
Obligatorio	Software	Entorno de desarrollo integrado o editor, por ejemplo Visual Studio Code, IntelliJ, NetBeans o Qt Creator.
Obligatorio	Software	Librería o framework para interfaz gráfica, por ejemplo Tkinter, PyQt, JavaFX, Swing o Qt.
Obligatorio	Software	Herramientas de control de versiones, preferiblemente Git y GitHub.

4.4 Entregables

A continuación, se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Documento	Especificación del lenguaje diseñado: tokens, palabras reservadas, operadores, reglas léxicas y gramática.
Código fuente	Implementación del analizador léxico.
Código fuente	Implementación del analizador sintáctico.

Código fuente	Interfaz gráfica funcional integrada con los analizadores.
Evidencia	Capturas o demostración del reconocimiento de tokens, generación del árbol sintáctico y manejo de errores.
Documento	Manual técnico con arquitectura, lógica de solución y decisiones de diseño.
Documento	Manual de usuario con instrucciones de instalación y uso.

5. Material de apoyo

Categoría	Link	Descripción
Documentación oficial / Ejemplo	https://ply.readthedocs.io	Documentación oficial de PLY (Python Lex-Yacc). Librería Python para implementar analizadores léxicos y sintácticos mediante expresiones regulares y gramáticas libres de contexto.
Libro / Documentación oficial	https://craftinginterpreters.com	Crafting Interpreters de Robert Nystrom. Guía completa y gratuita para construir un intérprete desde cero, cubriendo análisis léxico, sintáctico, AST y evaluación paso a paso.

6. Metodología

A continuación, se presenta una metodología sugerida para elaborar el presente proyecto:

Pasos a seguir:

1. Definir el lenguaje.

Establecer el propósito del lenguaje, sus construcciones principales, palabras

reservadas, operadores, identificadores, delimitadores y formato general de los programas.

2. **Especificar la fase léxica.**

Diseñar el conjunto de tokens y redactar las reglas léxicas necesarias para reconocer cadenas válidas y detectar errores.

3. **Implementar el analizador léxico.**

Programar el módulo que recorra el código fuente, separe lexemas, genere tokens y registre errores léxicos con línea y columna si es posible.

4. **Diseñar la gramática del lenguaje.**

Construir una gramática libre de contexto clara, consistente y suficiente para representar las estructuras elegidas del lenguaje.

5. **Implementar el analizador sintáctico.**

Desarrollar el componente que procese la secuencia de tokens, valide la estructura del programa y reporte errores sintácticos.

6. **Generar el árbol sintáctico.**

Diseñar una representación visual o estructurada del árbol de derivación para entradas correctas.

7. **Desarrollar la interfaz gráfica.**

Crear una GUI que permita ingresar código, ejecutar el análisis, visualizar tokens, árbol sintáctico y errores de forma ordenada.

8. **Realizar pruebas funcionales.**

Preparar casos de prueba válidos e inválidos para comprobar el comportamiento del sistema ante diferentes entradas.

9. **Documentar la solución.**

Elaborar la especificación del lenguaje, manual técnico y manual de usuario.

10. **Presentar y defender el proyecto.**

Mostrar el funcionamiento del sistema, justificar decisiones de diseño y explicar los resultados obtenidos.

Recomendaciones

- Iniciar con un lenguaje pequeño y bien definido antes de agregar complejidad.
- Probar primero el analizador léxico de forma independiente.
- Validar la gramática con ejemplos simples antes de integrarla al parser.
- Separar la lógica del análisis de la lógica de la interfaz gráfica.
- Mantener casos de prueba desde el inicio del proyecto.
- Usar control de versiones para registrar avances.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Definición del lenguaje y tokens	XXXX-XX-XX	XXXX-XX-XX
Implementación del analizador léxico	XXXX-XX-XX	XXXX-XX-XX

Diseño de gramática e implementación sintáctica	XXXX-XX-XX	XXXX-XX-XX
Desarrollo de interfaz gráfica	XXXX-XX-XX	XXXX-XX-XX
Pruebas, documentación y ajustes finales	XXXX-XX-XX	XXXX-XX-XX
Presentación del proyecto	XXXX-XX-XX	XXXX-XX-XX

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
Entrega puntual	Presentar el proyecto en la fecha establecida.	
Funcionamiento básico	El sistema debe ejecutar al menos el análisis léxico y sintáctico.	
Interfaz gráfica	Debe existir una interfaz funcional para interactuar con el sistema.	
Documentación	Debe entregarse manual técnico y manual de usuario.	

8.2 Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40	Desarrollo de competencias aplicadas durante el proyecto.	Cumple con autonomía, orden, documentación y presentación clara.	Presenta desorden, poca claridad o débil justificación técnica.	
1.1	Diseño del lenguaje	15	Descripción Calidad en la definición de tokens, reglas y coherencia del lenguaje.	El lenguaje está bien definido y es consistente.	El lenguaje presenta ambigüedades o reglas incompletas.	
			Rango del punteo			
1.2	Interfaz y presentación	25	Descripción Calidad de la GUI, usabilidad y demostración del proyecto.	La interfaz es funcional, clara y facilita el análisis.	La interfaz es limitada, confusa o incompleta.	
			Rango del punteo			

	Sub-Total de Puntos					
2	Conocimientos	60	Aplicación de conceptos técnicos de compiladores.	Evidencia dominio del análisis léxico y sintáctico.	Evidencia aplicación parcial o con errores importantes.	
2.1	Analizador léxico	25	Descripción Reconocimiento de tokens y manejo de errores léxicos.	Identifica correctamente tokens y errores.	Reconoce parcialmente o falla al manejar errores.	
			Rango del punteo			
2.2	Analizador sintáctico y árbol	35	Descripción Validación sintáctica, manejo de errores y generación del árbol.	Valida correctamente y genera árbol sintáctico funcional.	Presenta errores frecuentes o árbol incompleto/no funcional.	
			Rango del punteo			
	Sub-Total de Puntos					
	Total					

8.3 Comentarios Generales
